

Technical Discussion of `netcenter`

Network Construction, Shortest Paths, the 1-Median, and the Absolute 1-Center

Technical note for `netcenter` v0.1.0

24 September 2026

Abstract

`netcenter` is a Python package for solving classical facility-location problems on undirected spatial networks.

Given a road or other line-based network and an optional set of demand locations, the package computes:

1. a weighted network 1-median;
2. a vertex-restricted 1-center; and
3. an exact absolute 1-center whose optimum may lie in the interior of a network edge.

Distances are shortest-path distances on the constructed network rather than Euclidean distances in the plane. The facility-location mathematics is classical: the median and absolute-center formulation follows Hakimi, while the continuous center computation follows the edgewise structure developed in the network-location literature, particularly Kariv and Hakimi. The package does not claim a new location algorithm.

The package-specific work lies primarily in geospatial network construction, topology safeguards, numerical handling, memory management, vectorized implementation, pruning, parallel execution, and regression testing. This document describes both the mathematics and the implementation provenance of the first public release, v0.1.0, including the main pre-release corrections and engineering decisions.

1 Scope and notation

Let

$$G = (V, E)$$

be a connected undirected network with non-negative edge lengths.

Let

$$Q = \{q_1, \dots, q_k\} \subseteq V$$

be the unique demand vertices after external observations have been mapped to the network and duplicate locations have been coalesced.

Let

$$d_G(x, y)$$

denote shortest-path distance in the network.

Write

- $n = |V|$ for the number of network vertices;
- $m = |E|$ for the number of network edges;
- $k = |Q|$ for the number of unique demand vertices;
- L for the length of a particular edge.

The package currently solves three related location problems.

1.1 Weighted 1-median

Given non-negative demand weights w_i ,

$$x^* = \arg \min_{x \in V} \sum_{i=1}^k w_i d_G(x, q_i).$$

This is the network minimsum objective.

For vertex demand, Hakimi's vertex-optimality result implies that an optimal median may be chosen at a network vertex [9]. The implementation therefore evaluates network vertices rather than searching edge interiors.

1.2 Vertex 1-center

The vertex 1-center solves

$$x^* = \arg \min_{x \in V} \max_i d_G(x, q_i).$$

This is a node-restricted minimax objective.

1.3 Absolute 1-center

The absolute 1-center allows the candidate location to lie anywhere on the network:

$$x^* = \arg \min_{x \in G} \max_i d_G(x, q_i).$$

Unlike the median, an optimal absolute center need not occur at a vertex and may lie strictly inside an edge [9]. This edge-interior case is the principal mathematical reason for the package.

2 Algorithmic lineage

Every major mathematical component of **netcenter** is classical.

The principal references are:

- Hakimi [9], who formulated the absolute center and median of a graph and established the key vertex-optimality result used by the median solver;
- Hakimi [10], who extended the network-location framework to multiple facilities;
- Kariv and Hakimi [15], who developed algorithmic methods for network p -center problems, including the continuous absolute-center case;

- Kariv and Hakimi [16], for the corresponding p -median literature;
- Handler and Mirchandani [11], for a systematic treatment of location on networks and the classical edgewise center formulation;
- Daskin [4], for a modern treatment of network and discrete facility-location models.

Additional algorithms used as supporting infrastructure are also classical:

- Dijkstra’s shortest-path algorithm [5];
- k-d trees for nearest-neighbor lookup, due to Bentley [2];
- connected-component analysis using standard graph traversal, with the classical algorithmic lineage represented by Hopcroft and Tarjan [13];
- sweep-line segment-intersection algorithms such as Bentley–Ottmann [3], relevant to planar noding;
- finite-precision geometric techniques including Greene and Yao [8] and Hobby [12], relevant to robust geometric intersection and snap rounding.

The package does not reimplement all of these algorithms. Several are delegated to SciPy, Shapely, GEOS, and related numerical/geospatial libraries.

3 From spatial linework to a graph

A GIS line layer records geometry, not necessarily routing topology.

Two line geometries may cross in the plane without permitting movement between them. Conversely, a genuine junction may occur at a coordinate appearing in the interior of one or more LineStrings.

Accordingly, graph construction is part of the model, not merely a file-conversion step.

3.1 Default topology rule: shared-source-vertex noding

The default behavior is *shared-source-vertex noding*.

A source-vertex location is treated as a junction when vertices of at least two distinct input LineStrings lie there, or when a line endpoint lies there. The second condition includes a line’s own endpoint: a line that ends on one of its own interior vertices (for example, a cul-de-sac turning loop) is split where it touches itself. Every line is split at each of its interior vertices that lies at a junction.

This recovers common T-junctions and shared interior-interior junctions already encoded in source linework.

It does not create a new junction merely because two segments cross geometrically in two dimensions.

This distinction reduces the risk of false turns at:

- bridges;
- flyovers;

- tunnels;
- grade-separated intersections.

The shared-source-vertex rule is straightforward topology recovery rather than a new graph algorithm.

3.2 Endpoint-only mode

If the source is already segmented at every true junction, the caller may disable shared-vertex splitting.

In this mode, connectivity is inferred strictly from line endpoints.

3.3 Explicit planar noding

The caller may explicitly request full planar noding.

In that mode, geometric crossings are converted into junctions.

This is appropriate only when every two-dimensional crossing genuinely represents connectivity.

The sweep-line method for reporting all intersections among a set of segments is due to Bentley and Ottmann [3]. Practical finite precision intersection handling and snap rounding have their own computational geometry literature, including Greene and Yao [8] and Hobby [12].

netcenter does not implement those algorithms directly. Planar noding is delegated to Shapely/GEOS [21]. GEOS itself descends from the JTS Topology Suite.

3.4 Shape vertices and topology vertices

Road geometries often contain many intermediate coordinates whose purpose is only to describe curvature.

Those coordinates are geometrically useful but need not all survive as graph vertices.

Reducing unnecessary shape vertices matters because the shortest-path matrix scales with the number of graph nodes.

In the default and endpoint-only modes, an intermediate shape coordinate becomes a graph vertex only if it is a junction. Lines are not merged: where two input lines meet end to end and no third line meets them there, the meeting point remains a graph vertex of degree two. Line merging (Shapely/GEOS `line_merge`) runs only in the explicit planar-noding mode, after noding.

3.5 Matching near-identical coordinates

Endpoints intended to coincide may differ at very small floating-point scales.

Two coordinates closer than the snap distance (default 0.1 m) are treated as one network location, and the rule is applied transitively (single-linkage clustering, computed with a k-d tree). The same rule decides node identity, which source vertices are shared junctions, and whether a line is a ring. Within one line, two vertices are linked only when one of them is an endpoint and the line travels more than the snap distance between them, so the closely spaced shape vertices of a densely digitized road are not merged. This is an engineering convention rather than a named algorithm.

The first public release rounded coordinates to a grid with the snap spacing. Two copies of one junction that differ only by floating-point noise could then fall on opposite sides of a cell boundary and become two nodes, which cut the road. Node numbering still follows the grid cells, so node indices are the same as in that release.

3.6 Parallel edges

Real road data may contain multiple segments joining the same pair of graph vertices.

For shortest-path purposes, the sparse adjacency stores the minimum direct edge length between a node pair.

This is important because converting a sparse coordinate matrix to CSR can sum duplicate entries if not handled carefully. Summing parallel road lengths would be wrong and can violate the metric assumptions used by the center sweep.

3.7 Closed and nearly closed rings

A closed `LineString` has coincident start and end coordinates.

If treated naively, it can become a self-loop and disappear during graph cleaning.

A line whose two ends are one network location, under the rule above, is split at half its length before graph construction, so that the physical network remains represented.

3.8 Filtered slivers and orphan nodes

Small slivers or snapped self-loops may be removed after preliminary node IDs have been assigned.

The current implementation compacts node IDs after edge filtering so removed edges do not leave isolated matrix rows or columns that later produce spurious infinite distances.

3.9 Connected components

Real line datasets often contain isolated fragments.

Identifying connected components is standard graph traversal; the classical algorithmic lineage includes Hopcroft and Tarjan [13]. The package uses SciPy's sparse graph routines rather than implementing a component algorithm directly.

When component filtering is enabled, the retained component is selected by total road length rather than merely node count.

4 Coordinate systems and metric units

The mathematical optimization assumes meaningful edge lengths.

The package therefore operates in a projected coordinate reference system whose units are metres.

The policy is:

- projected metre-based input is preserved;
- geographic input is transformed to an inferred local UTM CRS unless an explicit target CRS is supplied;
- projected input using non-metre units is transformed;
- an explicitly supplied target CRS must be projected and metre-based.

For geographically large study regions, the analyst should supply a projection appropriate to the full study area rather than rely automatically on a single UTM zone.

5 Demand representation

External demand observations need not coincide exactly with graph vertices.

The current package maps each demand observation to the nearest network node.

For a point observation p_i ,

$$q_i = \arg \min_{v \in V} \|p_i - v\|_2.$$

The nearest-node lookup is performed with a k-d tree, using SciPy’s `ckdTree`; the k-d tree was introduced by Bentley [2].

A maximum snapping distance may be imposed.

This is an explicit modeling assumption: demand is represented at vertices, not continuously along edges.

5.1 Non-point geometries

If a demand layer contains polygons or other non-point geometries, each usable geometry is converted to a representative point before snapping.

5.2 Repeated snapped demand

Several observations may snap to the same network node.

For the minimax objectives, multiplicity has no effect:

$$\max\{d(x, v), d(x, v)\} = d(x, v).$$

Duplicate center demands can therefore be removed exactly.

For the median, multiplicity matters. Their weights are summed:

$$w_v = \sum_{i: q_i = v} w_i.$$

Thus duplicate-demand coalescing reduces shortest-path work without changing either optimization problem.

6 Shortest-path matrix

After demand coalescing, the package computes a shortest-path matrix

$$D \in \mathbb{R}^{k \times n},$$

where

$$D_{iv} = d_G(q_i, v).$$

Shortest paths are computed using Dijkstra’s algorithm [5] through SciPy’s sparse graph implementation [20].

The graph is required to be square, non-negative, and symmetric because the package currently models undirected networks.

Because symmetry is validated, SciPy’s Dijkstra routine is called in its directed mode. This gives identical distances and is faster, because the undirected mode builds a transposed copy of the graph on every call and relaxes each stored edge twice.

6.1 Computational cost

For k demand sources, n nodes, and m edges, a conventional binary-heap complexity statement is approximately

$$O(k(m + n \log n)),$$

although actual SciPy performance depends on implementation details and graph structure. The matrix itself requires

$$O(kn)$$

storage.

In many practical instances, memory usage of D is a more important limit than the arithmetic required by the location objective.

6.2 Chunking

Dijkstra calculations are performed in bounded groups of source nodes.

If a block contains r demand sources, its temporary output contains approximately rn floating-point values.

The `max_temp_mb` parameter controls the target size of one temporary float64 block per worker.

When several workers are requested, the sources are also split into at least as many blocks as there are workers, so that every worker receives work even when all rows would fit in one memory block.

6.3 Streaming parallel assembly

When several Dijkstra blocks are computed in parallel, completed blocks are written directly into a preallocated final matrix.

This avoids retaining every completed block and then constructing another full matrix by concatenation.

6.4 Reduced precision

The final shortest-path matrix may optionally be stored in float32.

Approximate storage requirements are:

$$8kn \text{ bytes}$$

for float64 and

$$4kn \text{ bytes}$$

for float32.

Float32 is a memory–precision trade-off. The optimization remains internally consistent with the stored matrix, but the stored metric is itself rounded relative to float64 output.

The median objective is accumulated in float64 even when the matrix is stored in float32, because a float32 running sum over thousands of demand rows loses metres of precision. Weighted sums are evaluated in bounded row blocks, so the float32 matrix is never converted to float64 in one piece.

7 Weighted 1-median

For every candidate node v , define

$$M(v) = \sum_{i=1}^k w_i D_{iv}.$$

The weighted network median is

$$v^* = \arg \min_{v \in V} M(v).$$

Hakimi's vertex-optimality result reduces the continuous network median problem to this finite evaluation for vertex demand [9].

In matrix notation, if

$$w = \begin{bmatrix} w_1 & \cdots & w_k \end{bmatrix},$$

then

$$M = wD.$$

The implementation is therefore a weighted reduction over the shortest-path matrix followed by an **argmin**.

If no explicit weights are supplied, unit weights are used.

With unit weights, minimizing the total distance is closely related to maximizing closeness centrality, whose classical lineage includes Bavelas [1] and Freeman [6].

On trees, related median and center problems admit simpler algorithms; Goldman [7] is a classical reference.

8 Vertex 1-center

For each candidate vertex v , define eccentricity relative to the selected demand set:

$$E(v) = \max_i D_{iv}.$$

The vertex center is

$$v^* = \arg \min_{v \in V} E(v).$$

Computationally, this is a columnwise maximum over D , followed by an **argmin**.

The resulting radius

$$R_V = \min_{v \in V} \max_i D_{iv}$$

also provides a valid incumbent for the continuous absolute-center search.

On trees, the classical graph center goes back to Jordan [14].

9 Absolute 1-center

The absolute-center problem requires searching edge interiors.

Consider edge

$$e = (u, w)$$

with length L .

Parameterize a point $x(t)$ on the edge by distance

$$t \in [0, L]$$

from endpoint u .

For demand q_i , define

$$a_i = d_G(q_i, u), \quad b_i = d_G(q_i, w).$$

Any route from the interior point to the remainder of the graph must leave the edge through one of its endpoints.

Therefore

$$d_G(q_i, x(t)) = \min\{a_i + t, b_i + L - t\}. \quad (1)$$

This is a tent-shaped piecewise-linear function.

9.1 Switch point

The two endpoint routes are equal when

$$a_i + t = b_i + L - t.$$

Thus the switch point is

$$t_i^* = \frac{b_i - a_i + L}{2}. \quad (2)$$

Because u and w are connected by an edge of length L ,

$$|a_i - b_i| \leq L$$

by the triangle inequality, hence

$$t_i^* \in [0, L]$$

in exact arithmetic.

9.2 Upper envelope

The eccentricity of $x(t)$ is

$$E_e(t) = \max_i d_G(q_i, x(t)).$$

Sort the switch points:

$$t_{(1)}^* \leq t_{(2)}^* \leq \dots \leq t_{(k)}^*.$$

Between consecutive switch points, the sets of demands using the two endpoint branches remain fixed.

On one such interval,

$$E_e(t) = \max\{A + t, B + L - t\},$$

where A is the largest a_i among demands still using the u -side branch, and B is the largest b_i among demands using the w -side branch.

The first term rises with slope $+1$ and the second falls with slope -1 .

Their unconstrained intersection occurs at

$$\hat{t} = \frac{B + L - A}{2}.$$

For interval

$$[t, \bar{t}],$$

the interval minimizer is

$$t^* = \text{clip}(\hat{t}; t, \bar{t}).$$

Evaluating all breakpoint intervals yields the exact minimum on the edge, modulo floating-point precision.

9.3 Prefix and suffix maxima

After sorting the switch points, the implementation computes:

- suffix maxima of the a_i ;
- prefix maxima of the b_i .

This gives all values of A and B in linear work after sorting.

The resulting per-edge complexity is

$$O(k \log k),$$

dominated by sorting.

This is a direct computational realization of the classical absolute-center edge structure associated with Hakimi and Kariv–Hakimi [9, 15], with textbook treatment in Handler and Mirchandani [11] and Daskin [4].

The particular NumPy prefix/suffix organization is an implementation choice, not a separate location algorithm.

9.4 Related literature not implemented

Minieka [18] studied the related m -center problem and continuous-network variants.

Megiddo [17] later developed parametric-search techniques that remove logarithmic factors in related network-center computations. Those refinements are not implemented here.

10 Safe edge pruning

Sweeping every edge is often unnecessary.

The vertex-center radius

$$R_V = \min_{v \in V} \max_i D_{iv}$$

is already a feasible absolute-center solution.

For any point $x(t)$ on edge (u, w) ,

$$d_G(q_i, x(t)) = \min\{D_{iu} + t, D_{iw} + L - t\}.$$

Because $t \geq 0$ and $L - t \geq 0$,

$$d_G(q_i, x(t)) \geq \min\{D_{iu}, D_{iw}\}.$$

Therefore every point on the edge satisfies

$$E_e(t) \geq LB_e := \max_i \min\{D_{iu}, D_{iw}\}. \quad (3)$$

If LB_e cannot improve the incumbent radius, the edge need not be swept.

A second, cheaper bound uses only the vertex eccentricities

$$E(v) = \max_i D_{iv},$$

which are already computed for the vertex center. By the triangle inequality, $D_{iu} \leq t + d_G(q_i, x(t))$, so $E_e(t) \geq E(u) - t$. In the same way, $E_e(t) \geq E(w) - (L - t)$. The larger of the two right-hand sides is smallest where they are equal, so for every $t \in [0, L]$

$$E_e(t) \geq LB'_e := \frac{1}{2}(E(u) + E(w) - L). \quad (4)$$

LB'_e costs $O(1)$ per edge, while LB_e costs $O(k)$ per edge. The implementation evaluates LB'_e for every edge and LB_e only for the edges that pass LB'_e . An edge is swept only if both bounds are below the incumbent radius plus the numerical margin. Both are valid lower bounds, so the result is unchanged. On the road-like test networks used during development, LB'_e alone left about five of roughly 18,600 edges.

This is exact branch-and-bound logic in real arithmetic.

It is an implementation acceleration rather than a different facility-location objective.

11 Numerical safeguards

The mathematical algorithms operate over exact real-valued network distances, while the implementation uses floating-point arithmetic.

Several safeguards are therefore applied.

11.1 Breakpoint clipping

The triangle inequality should place every switch point in $[0, L]$.

Tiny numerical violations are defensively clipped:

$$t_i^* \leftarrow \text{clip}(t_i^*; 0, L).$$

11.2 Pruning tolerance

When the distance matrix uses reduced precision, a lower bound may differ slightly from its exact value.

A dtype- and scale-aware margin is included when comparing lower bounds with the incumbent radius so a candidate edge is not discarded solely because of floating-point rounding.

11.3 Meaning of exactness

The absolute-center sweep is algorithmically exact for the supplied shortest-path matrix.

Thus, “exact” means:

exact for the stored network metric, modulo explicit floating-point tolerance.

A float32 run is therefore exact with respect to the float32 distance matrix, but that matrix is itself a reduced-precision approximation to the corresponding float64 metric.

12 Parallel execution and memory management

The package separates mathematical computation from execution strategy.

Parallelism does not change the mathematical objective or intended result.

12.1 Shortest-path blocks

Demand sources are divided into bounded blocks.

Each block is passed to SciPy’s Dijkstra routine.

Results are written directly into a preallocated final matrix.

12.2 Absolute-center blocks

The edge sweep operates on bounded groups of segment–demand cells.

The `max_cells` parameter controls peak temporary storage.

12.3 Worker policy

One worker is the conservative default.

Additional workers may be requested explicitly.

The package exposes process- and thread-based backends because their performance and memory trade-offs depend on:

- graph size;
- demand count;

- operating system;
- SciPy build;
- available RAM;
- memory bandwidth.

No backend is assumed to be universally superior.

13 Complete computational pipeline

The first public release can be summarized as follows.

```
BUILD-NETWORK(linework, topology options):

    project linework to a metre-based CRS
    remove unusable geometries

    if planar noding requested:
        split at geometric crossings

    else if shared-vertex noding enabled:
        split where source vertices are shared
        by distinct input lines

    preserve closed rings
    identify network nodes
    remove slivers and self-loops
    compact unused node IDs

    create undirected sparse adjacency
    retain minimum direct weight for parallel node pairs

    optionally restrict to a connected component

    return network

PREPARE-DEMAND(demand):

    convert non-point geometries to representative points
    transform to network CRS
    snap each observation to nearest network node
    enforce optional maximum snap distance
    coalesce duplicate snapped nodes

    return unique demand nodes and median weights

SOLVE(network, demand):

    D <- shortest-path distances from demand nodes
        to all network vertices using Dijkstra
```

```

median <-
  argmin_v sum_i w_i D[i,v]

vertex_center <-
  argmin_v max_i D[i,v]

incumbent <- vertex_center

for each edge e=(u,w,L):

  LB[e] <-
    max_i min(D[i,u], D[i,w])

discard edges that cannot improve incumbent

for each surviving edge:

  a_i <- D[i,u]
  b_i <- D[i,w]

  switch_i <-
    (b_i - a_i + L) / 2

  sort demands by switch_i

  compute suffix maxima of a_i
  compute prefix maxima of b_i

  analytically minimize
    max(A + t, B + L - t)
  on every breakpoint interval

absolute_center <-
  best of vertex incumbent
  and surviving edge-interior candidates

return median,
  vertex_center,
  absolute_center

```

14 Complexity

A useful high-level complexity summary is:

Stage	Indicative cost	Source / comment
Planar intersection reporting	$O((s + i) \log s)$	Bentley–Ottmann for s segments and i intersections.
Nearest-node demand mapping	approximately $O(k \log n)$	k-d tree; behavior depends on data distribution.
Shortest-path matrix	approximately $O(k(m + n \log n))$	Dijkstra; often the dominant stage.
Weighted 1-median	$O(kn)$	Hakimi vertex reduction followed by weighted column sum.
Vertex 1-center	$O(kn)$	Columnwise maximum followed by minimum.
Edge lower bounds	$O(m + km')$	LB'_e on all m edges; LB_e only on the m' survivors.
Absolute-center sweep	$O(mk \log k)$ worst case	Only unpruned edges are swept in practice.

The shortest-path matrix requires $O(kn)$ memory and is frequently the practical bottleneck.

15 Implementation work beyond the published algorithms

The mathematical objectives and their classical structure are not novel.

The following implementation choices were developed during pre-release work, including AI-assisted development sessions. They should be understood as engineering on top of the published algorithms rather than as new facility location methods.

15.1 Optimizations

1. Segment pruning by a lower bound.

The bound

$$LB_e = \max_i \min\{D_{iu}, D_{iw}\}$$

is used to eliminate edges that cannot improve the best known vertex radius.

2. Batched vectorization of the sweep.

Many candidate edges are solved simultaneously using NumPy arrays. Prefix and suffix running maxima replace repeated interval-by-interval loops.

3. Memory-budgeted sweep blocking.

The absolute-center sweep processes bounded groups of segment–demand cells so temporary memory does not scale directly with the entire edge set.

4. Contiguous sort inputs.

Endpoint-distance arrays are copied into row-contiguous blocks before per-edge sorting to simplify and accelerate vectorized operations.

5. Conservative parallelism.

Worker count is treated as a maximum, not a promise. Small workloads remain serial when worker startup would dominate useful work.

6. Bounded shortest-path chunks.

Dijkstra source nodes are processed in bounded row groups and cast promptly to the requested storage dtype.

7. Streaming parallel assembly.

Parallel Dijkstra results are consumed in order and written directly into the final matrix rather than accumulated and stacked afterward.

8. Exact demand coalescing.

Repeated observations snapped to the same node are solved once. Counts or weights are preserved exactly for the median, while duplicates vanish from the minimax objective.

9. Scale-aware pruning tolerance.

The pruning margin widens according to the stored floating-point dtype so a borderline edge is not removed merely because of rounding.

15.2 An optimization tried and rejected

A best-first bound-tightening strategy was explored during pre-release development.

The idea was to sweep the most promising edges first, update the incumbent radius, and then repeat pruning with the tighter bound.

This is standard branch-and-bound practice.

It was removed because measurements showed that the first lower-bound pass already reduced the candidate edge set sufficiently that the second pruning stage did not justify the added complexity.

This history is retained to prevent future optimization work from repeating the same experiment without measurement.

16 Correctness bugs found and fixed during pre-release development

Several important failures were identified before the first public release. These are worth documenting because they could produce plausible wrong answers rather than obvious crashes.

16.1 Parallel edges silently summed

Sparse COO-to-CSR conversion can add duplicate entries.

If two roads join the same pair of graph nodes and their lengths are summed, the resulting direct edge weight may become larger than a valid alternative path.

This can violate the metric structure expected by the absolute-center sweep.

The current implementation collapses parallel edges using the minimum direct length.

16.2 Closed loops deleted

A closed road may merge into a LineString whose endpoints coincide.

If the result is then interpreted as a self-loop and removed, the entire ring disappears from the graph.

The current graph builder splits such rings before self-loop filtering.

16.3 Empty geometries corrupting endpoint lookup

Empty geometries can produce invalid coordinate runs and incorrect endpoint association if not removed before indexing.

The graph builder filters unusable geometries before endpoint extraction.

16.4 Needless reprojection

Automatically reprojecting all input to a locally inferred UTM zone is appropriate for geographic coordinates but can be wrong for input already projected in valid metre units.

The current implementation preserves projected metre-based input and only reprojects when necessary.

16.5 Nearly closed rings deleted

A ring can be visually closed while its first and last coordinates differ by floating-point noise.

Exact closure tests may miss this case.

The graph builder now decides ring closure with exactly the rule used for node identity: a line is a ring when both ends are one network location. In the first public release the two decisions still used different rules; see Section 17.

16.6 False junctions at grade-separated crossings

Treating every two-dimensional line crossing as connected can create impossible turns between bridges, flyovers, tunnels, and roads below them.

Full planar noding is therefore explicit rather than the default.

16.7 Shared-vertex noding initially too narrow

An earlier pre-release rule handled T-junctions where one line ended on the interior of another but still missed cases where two lines both continued through a shared source vertex.

The current rule defines a junction by participation of at least two distinct input LineStrings at a source-vertex location, or by a line endpoint at that location.

16.8 Filtered slivers leaving orphan graph nodes

Assigning node IDs before removing slivers and snapped self-loops can leave unused rows and columns in the sparse graph.

The current implementation compacts node IDs after edge filtering.

17 Changes after the first public release

A code review in September 2026, again AI-assisted, found the defects below in v0.1.0. Each was reproduced before it was fixed, and each now has a regression test. The package version number was not changed.

17.1 Junctions split at grid-cell boundaries

Node identity rounded coordinates to a grid. A junction coordinate on a cell boundary, such as a centimetre coordinate ending in 5 with the default 0.1 m snap, was assigned to one cell or the other by floating-point noise of about 10^{-9} m. The two copies of one junction then became two nodes and the road was cut. Node identity now uses the distance rule of Section 3.5.

17.2 Rings deleted when closure and identity disagreed

Ring closure was tested by Euclidean distance (at most the snap distance), but endpoints were merged by grid cell. Ends up to $\sqrt{2} \times 0.1$ m apart in one cell became one node without passing the closure test, so the ring became a self-loop and was deleted. With a 0.113 m end gap, a 1.9 km ring disappeared. A line is now a ring when both ends are one network location.

17.3 Lines that end on themselves

A line that ends on one of its own interior vertices, such as a cul-de-sac turning loop drawn as one line, was not split there, because a shared vertex required two distinct lines. In a test, the road distance from the loop entrance to the loop junction was 500 m instead of 100 m.

17.4 Failure with NumPy 2.0.0

NumPy 2.0.0, which the declared requirement `numpy>=1.24` allows, returns a two-dimensional inverse from `numpy.unique` with `axis=0` and `return_inverse=True`. The shared-vertex step did not flatten it, so graph construction failed for every input with two or more lines. The continuous-integration workflow installs only the newest versions, so it could not detect this.

17.5 Float32 median memory and precision

With float32 storage, the weighted median evaluated `weights @ D`, which converts the whole matrix to a float64 copy: for a 92 MiB matrix, 184 MiB of extra memory, more than float64 storage itself would have used. Without weights, column sums were accumulated in float32. For 2,000 demand points and 12,000 nodes, the reported objective was 1.9 m too small, and objectives at other nodes were off by up to 58.9 m in either direction; in that test the chosen node was still correct. Both sums are now accumulated in float64, in bounded row blocks.

17.6 Parallel shortest paths not used

Sources were split into blocks by memory only. With the default 256 MiB budget, a $2,000 \times 12,000$ problem fits in one block, so it ran serially whatever number of workers was requested.

17.7 Command-line error reporting

GeoPandas 1.x reads and writes files through pyogrio, whose errors derive from `RuntimeError`, which the command-line interface did not catch. A mistyped input path therefore produced a Python traceback instead of a one-line error message.

17.8 Optimizations

- The eccentricity bound LB'_e of Equation (4): the absolute-center stage took 0.31 s before and 0.03 s after for 2,000 demand points and 12,000 nodes, and 2.1 s before and 0.24 s after with every node as demand.
- Directed Dijkstra on the validated symmetric graph: about 10% faster, with identical distances.
- Vectorized line cutting in shared-vertex nodding: graph construction took 2.5 s before and 0.8 s after on OSM-shaped input with 160,000 junctions.
- Distance-matrix validation by minimum and maximum instead of full-size boolean masks.

All timings were measured on one CPU core. On inputs without the defects above, the results are identical to v0.1.0: the same nodes, node numbering, edges, and result locations, with objective values equal to within 10^{-15} relative.

18 Validation strategy

Validation separates facility-location mathematics from GIS graph-construction correctness as much as possible.

18.1 Hand-computable cases

Small networks with known answers test:

- weighted median locations;
- vertex centers;
- edge-interior absolute centers.

18.2 Independent edge sampling

The continuous absolute-center solution is compared with independent brute-force sampling along network edges on randomly generated test networks.

The sampled solution is approximate, but it shares no breakpoint logic with the exact sweep. It therefore provides an independent check of the sweep implementation.

18.3 What brute force does not validate

Brute-force edge sampling validates the optimizer conditional on the supplied graph and shortest-path matrix.

It does not validate whether the graph correctly represents the intended road system.

For example, no numerical test can determine from geometry alone whether a bridge should connect to a road below it.

That remains a topology and data-semantics question.

18.4 Regression coverage

Regression tests cover failures including:

- parallel edges being combined incorrectly;
- closed rings disappearing as self-loops;
- nearly closed rings being lost because of floating-point differences;
- empty geometries corrupting node assignment;
- false connections at geometric bridge crossings;
- missed T-junctions;
- missed shared interior-interior junctions;
- filtered edges leaving isolated graph nodes;
- invalid projected units;
- disconnected demand/network combinations;
- worker-count and block-size invariance;
- float32 pruning behavior;
- package-version and CLI consistency;
- junctions split by floating-point noise at a grid-cell boundary;
- rings deleted because ring closure and node identity used different rules;
- lines that end on one of their own interior vertices;
- float32 median memory use and summation precision;
- parallel Dijkstra requests that fit in one memory block;
- CLI error messages for unreadable input and unwritable output;
- agreement of the two pruning bounds with an unpruned sweep.

19 Pre-release development history

Before v0.1.0, the codebase underwent several internal development and review iterations.

Those intermediate states were not intended to define a public semantic-version sequence and are therefore described here simply as *pre-release development history*.

The important distinction is between:

1. changes to the mathematical facility-location problem; and
2. changes to the graph, numerical representation, and implementation on which that problem is solved.

The core optimization problems remained the same throughout pre-release development:

- weighted vertex 1-median;
- vertex-restricted minimax center;
- continuous absolute 1-center.

Most substantive pre-release changes instead concerned:

- safer road-network topology;
- correct treatment of parallel edges;
- preservation of rings;
- coordinate-system handling;
- duplicate-demand reduction;
- bounded-memory shortest paths;
- numerical validation;
- parallel execution;
- regression coverage.

These changes can alter a numerical answer when they alter the graph G being represented, but they do not define a different median or center objective.

20 Audit: same mathematics versus changed preprocessing

The table below summarizes the distinction.

Aspect	Early prototype	Current code	Changed mathematics?
Weighted 1-median	Weighted total shortest-path distance over vertices.	Same objective and vertex reduction.	No.
Vertex 1-center	Maximum demand distance minimized over vertices.	Same objective and computation.	No.
Absolute 1-center	Continuous edge-interior minimax problem using breakpoint structure.	Same mathematical problem and same edgewise envelope structure.	No.
Shortest paths	Dijkstra on an undirected sparse graph.	Same shortest-path metric, with stronger validation and bounded-memory execution.	No.
Edge pruning	Lower bound based on endpoint distances.	Same bound, preceded by a cheaper eccentricity bound, with more conservative floating-point handling.	No.

Aspect	Early prototype	Current code	Changed mathematics?
Demand duplicates	Could produce repeated shortest-path rows.	Coalesced exactly before shortest-path computation.	No.
Crossing treatment	Earlier prototype could planarize all crossings.	Shared-source-vertex noding is default; full planar noding is explicit.	Preprocessing changed.
Shared interior junctions	Not fully recovered in the earliest rule.	Recovered when distinct source LineStrings share a source-vertex location, or a line ends on a vertex of another line or of itself.	Preprocessing changed.
Connected component choice	Could depend on node count.	Uses total road length.	Preprocessing changed.
CRS handling	Weaker assumptions about projected units.	Metre-based projected CRS required or constructed explicitly.	Measurement policy changed.
Parallel-edge handling	Duplicate sparse entries could sum.	Minimum direct edge length retained.	Bug fix.
Closed rings	Could disappear as self-loops.	Preserved by splitting before graph construction; closure uses the node-identity rule.	Bug fix.
Unused node IDs	Could survive edge filtering.	Compacted after filtering.	Bug fix.
Precision	Primarily float64-oriented.	float32 storage optional with safe pruning tolerance; median sums accumulated in float64.	Optional numerical approximation only.

The most important point is that the location objective and the classical absolute-center method remained stable. The pre-release changes mostly improved the graph to which those algorithms are applied.

21 Scope and limitations

The first public release deliberately addresses a restricted class of network-location problems.

21.1 Undirected networks

The graph is undirected.

The package does not currently model:

- one-way streets;
- asymmetric travel costs;
- turn restrictions;
- time-dependent routing.

These require a different state representation and, in particular, a reconsideration of continuous edge-interior treatment.

21.2 Vertex demand

External demand is snapped to graph vertices.

Demand is not currently represented continuously along edge interiors.

21.3 Unweighted minimax objective

Weights affect the 1-median but not the center.

The current center objective is

$$\min_x \max_i d_G(x, q_i),$$

not a weighted minimax variant.

A weighted minimax objective changes the edgewise functions and cannot simply reuse the current prefix/suffix structure unchanged.

21.4 One facility

The package solves the $p = 1$ case.

General p -center and p -median problems are substantially more difficult and belong to a broader optimization literature [15, 16].

21.5 Static edge costs

Edge costs represent a fixed metric.

Congestion, traffic variation, schedules, stochastic travel time, and other dynamic costs are outside the present scope.

21.6 Topology semantics

Geometry alone cannot fully determine real transportation connectivity.

The graph builder does not automatically infer every bridge, tunnel, layer, access, one-way, or turn-restriction semantic from external road-database attributes.

The analyst remains responsible for ensuring that the input network represents the intended movement system.

22 Network median versus geographic centroid

The network median should not be confused with a Euclidean centroid.

For Euclidean coordinates x_i , a weighted arithmetic centroid is

$$\bar{x} = \frac{\sum_i w_i x_i}{\sum_i w_i}.$$

Equivalently, it minimizes a squared Euclidean-distance objective.

The network median instead minimizes

$$\sum_i w_i d_G(x, q_i).$$

It is therefore more naturally interpreted as a *minisum accessibility center* than as a literal coordinate centroid.

A closer network analogue of a Euclidean squared-distance centroid would be a Fréchet mean or network barycenter:

$$x^* = \arg \min_{x \in G} \sum_i w_i d_G(x, q_i)^2.$$

That problem is not implemented in v0.1.0.

23 Implementation versus mathematical contribution

It is important to distinguish published facility-location mathematics from the package implementation.

The following are classical:

- the network median objective;
- the absolute-center objective;
- vertex optimality of the network median for vertex demand;
- edge-interior absolute centers;
- shortest-path distance as the network metric;
- the breakpoint structure of the continuous center problem.

The following are implementation and engineering choices in **netcenter**:

- GIS linework preprocessing;
- shared-source-vertex topology recovery;
- defensive handling of rings, slivers, and parallel edges;
- demand coalescing;
- NumPy vectorization;
- prefix/suffix envelope evaluation;
- lower-bound pruning;
- blocked shortest-path computation;
- streaming parallel assembly;
- reduced-precision storage support;
- numerical tolerance policy;
- regression-test coverage.

No novelty claim is made for the underlying network-location algorithms.

24 Conclusion

`netcenter` v0.1.0 is best understood as a practical geospatial implementation of classical network-location theory.

Its mathematical core is compact:

1. construct a metric graph from linework;
2. map demand to network vertices;
3. compute shortest-path distances;
4. solve the weighted 1-median directly over vertices;
5. solve the vertex minimax center directly over vertices;
6. use that center as an incumbent;
7. prune edges whose lower bounds cannot improve the incumbent;
8. solve the remaining edge-interior minimax problems analytically through breakpoint sorting and upper-envelope minimization.

The principal difficulty in practical use is not defining the objective but ensuring that the graph faithfully represents the intended transportation network.

Accordingly, much of the package's engineering is devoted to topology, coordinate systems, memory safety, numerical safeguards, provenance, and validation rather than to modifying the classical facility-location mathematics.

References

- [1] A. Bavelas. Communication patterns in task-oriented groups. *Journal of the Acoustical Society of America*, 22(6):725–730, 1950.
- [2] J. L. Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
- [3] J. L. Bentley and T. A. Ottmann. Algorithms for reporting and counting geometric intersections. *IEEE Transactions on Computers*, C-28(9):643–647, 1979.
- [4] M. S. Daskin. *Network and Discrete Location: Models, Algorithms, and Applications*. 2nd edition, Wiley, 2013. doi:10.1002/9781118537015.
- [5] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [6] L. C. Freeman. Centrality in social networks: conceptual clarification. *Social Networks*, 1(3):215–239, 1978.
- [7] A. J. Goldman. Optimal center location in simple networks. *Transportation Science*, 5(2):212–221, 1971.

- [8] D. H. Greene and F. F. Yao. Finite-resolution computational geometry. In *27th Annual Symposium on Foundations of Computer Science*, pages 143–152, 1986.
- [9] S. L. Hakimi. Optimum locations of switching centers and the absolute centers and medians of a graph. *Operations Research*, 12(3):450–459, 1964. doi:10.1287/opre.12.3.450.
- [10] S. L. Hakimi. Optimum distribution of switching centers in a communication network and some related graph theoretic problems. *Operations Research*, 13(3):462–475, 1965. doi:10.1287/opre.13.3.462.
- [11] G. Y. Handler and P. B. Mirchandani. *Location on Networks: Theory and Algorithms*. MIT Press, Cambridge, Massachusetts, 1979.
- [12] J. D. Hobby. Practical segment intersection with finite precision output. *Computational Geometry*, 13(4):199–214, 1999.
- [13] J. Hopcroft and R. Tarjan. Algorithm 447: efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378, 1973.
- [14] C. Jordan. Sur les assemblages de lignes. *Journal für die reine und angewandte Mathematik*, 70:185–190, 1869.
- [15] O. Kariv and S. L. Hakimi. An algorithmic approach to network location problems. I: The p -centers. *SIAM Journal on Applied Mathematics*, 37(3):513–538, 1979. doi:10.1137/0137040.
- [16] O. Kariv and S. L. Hakimi. An algorithmic approach to network location problems. II: The p -medians. *SIAM Journal on Applied Mathematics*, 37(3):539–560, 1979. doi:10.1137/0137041.
- [17] N. Megiddo. Applying parallel computation algorithms in the design of serial algorithms. *Journal of the ACM*, 30(4):852–865, 1983.
- [18] E. Minieka. The m -center problem. *SIAM Review*, 12(1):138–139, 1970.
- [19] P. B. Mirchandani and R. L. Francis, editors. *Discrete Location Theory*. Wiley, 1990.
- [20] SciPy developers. `scipy.sparse.csgraph.dijkstra` documentation. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.csgraph.dijkstra.html>.
- [21] Shapely developers. `shapely.node` documentation. <https://shapely.readthedocs.io/en/stable/reference/shapely.node.html>.